Ben-Gurion University of the Negev
Faculty of Engineering Sciences
Department of Software and Information Systems Engineering

Deep Reinforcement Learning

Assignment #1

Due date: 26/11/2025

Lecturer: Dr. Gilad Katz
Students: Mark Feldman (320827637), Tamir Levin (315765347)

Question 1: Why methods such as Value-Iteration cannot be implemented in such environments? Write down the main problem.

Value-Iteration requires full knowledge of the environment's dynamics in order to compute the expected return of each state–action pair. Specifically, it depends on the transition probabilities $p(s'|s,a)$. In model-free environments, these functions are unknown or cannot be explicitly provided to the agent. Therefore, algorithms like Value-Iteration, which rely on a known model of the environment, cannot be directly applied, motivating the use of model-free methods such as Q-learning instead.

Question 2: How do model-free methods resolve the problem you wrote in the previous question? Explain shortly.

Model-free methods address the lack of knowledge about the environment's dynamics by learning through direct interaction with it. Instead of relying on the transition probabilities $p(s'|s,a)$ and the reward function $r(s,a)$, they estimate the state–action value function $q(s,a)$ by sampling experiences. Through repeated exploration and updates, the agent learns which actions yield higher overall returns. While the value function $v(s)$ can also be estimated from samples, it only indicates how good a state is on average, whereas $q(s,a)$ provides information about each possible action in a state, allowing the agent to choose the optimal action for each state directly.

Question 3: What is the main difference between SARSA and Q-learning algorithms? Explain shortly the meaning of this difference.

The main difference between SARSA and Q-learning is the way they update the Q-value. Q-learning is an off-policy method, meaning it learns the optimal policy independently of the agent's current behavior. It updates the Q-value using the greedy action that maximizes the estimated return in the next state. SARSA, on the other hand, is an on-policy method that updates the Q-value based on the actual action chosen by the agent in the next step. In other words, Q-learning aims to learn the optimal policy regardless of how the agent currently behaves, while SARSA learns the value of the policy the agent is actually following, making it more sensitive to the exploration strategy.

Question 4: Why is it better than acting greedily (choosing an action with $\arg\max_a Q(S', a)$)?

When the transition dynamics of the environment are unknown, as in model-free reinforcement learning, the agent must explore through trial and error to discover which actions yield the highest overall rewards. If the agent always acts greedily by selecting only the action with the highest estimated value, it may fail to explore other potentially better actions and converge to a suboptimal policy. To avoid this, the agent follows an ε-greedy strategy, where with a small probability ε it explores random actions, and with probability $1 - \varepsilon$ it exploits the current best-known action. A common approach is to gradually decrease ε during training to balance exploration and exploitation effectively. In this approach, at the beginning of training (t_0), the agent explores more to learn about the environment, while at later stages ($t \to T$), after most states have been visited, exploration is reduced to focus on exploiting the learned knowledge.

Q-Learning Implementation

To solve the FrozenLake-v1 problem, we implemented the Tabular Q-Learning algorithm using a ϵ-greedy exploration policy and a decay schedule. The environment has 16 discrete states and 4 actions, therefore the Q-function is represented as a 16×4 table updated online after every transition.

Hyper-Parameter Optimization

To identify a stable and well-performing configuration, we performed a sweep over:

$$\alpha \in \{0.1, 0.01, 0.001\}, \qquad \gamma \in \{0.99, 0.995, 0.999\},$$

$$\epsilon_{\text{decay}} \in \{0.99, 0.999\}, \qquad \text{schedule} \in \{\text{linear}, \text{exp}\}.$$

Each configuration was trained for 5000 episodes and evaluated using the mean reward over the last 100 episodes as shown in Figure 1. We highlight the top few configurations for clarity, while the remaining curves appear in grey.

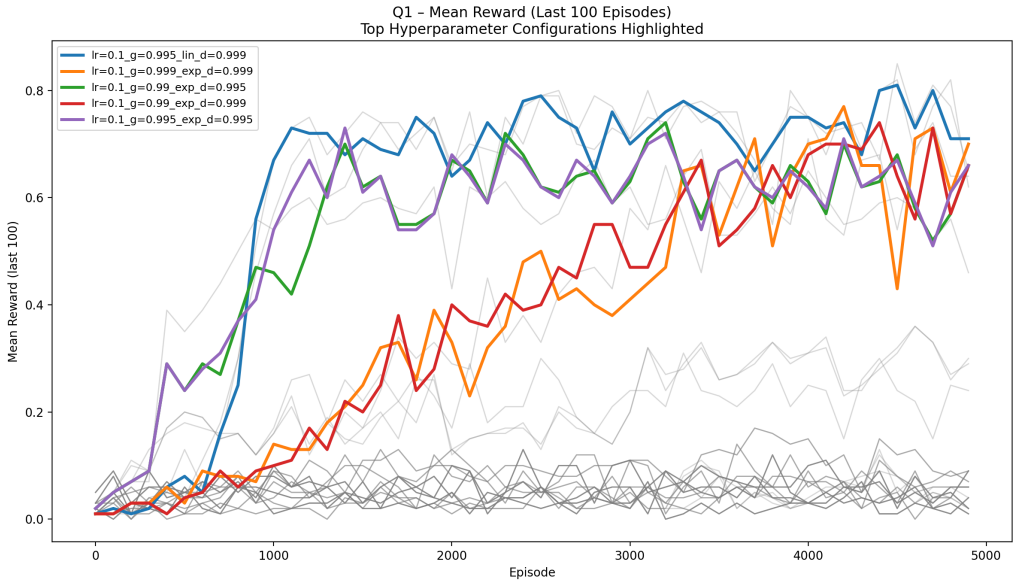


Figure 1: Hyperparameter sweep.

Based on the sweep, the following parameters achieved the most stable convergence and highest average reward:

$$\alpha = 0.1, \qquad \gamma = 0.999, \qquad \epsilon\text{-schedule} = \text{linear}, \qquad \epsilon_{\text{decay}} = 0.999.$$

This configuration was used for the final training run of 5000 episodes. Figures 2 and 3 show the reward per episode and the mean number of steps to reach the goal, averaged over the last 100 episodes. As learning progresses, the agent consistently reaches the goal in fewer steps while increasing its average reward.

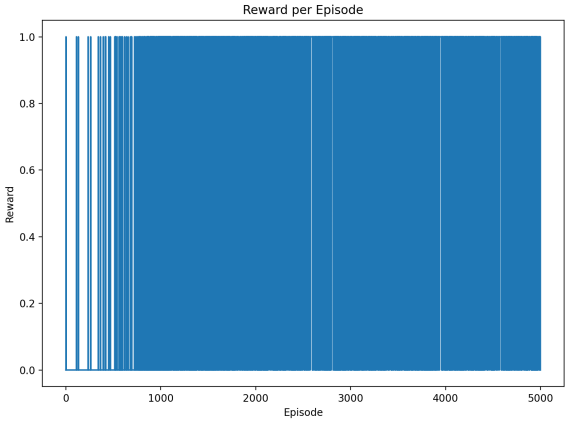
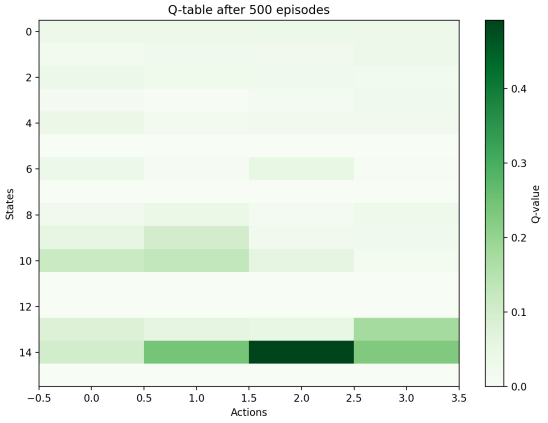


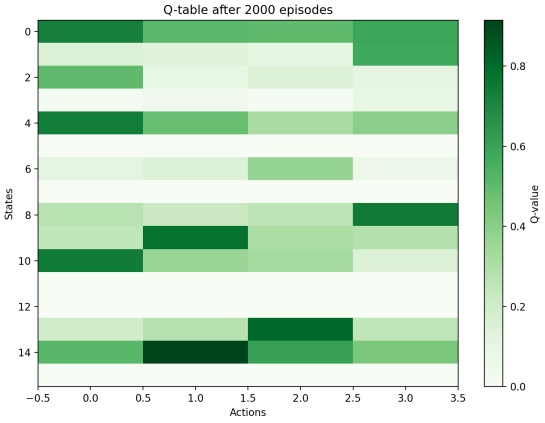
Figure 2: Reward per episode during training.

Figure 3: Mean steps to goal over the last 100 episodes.

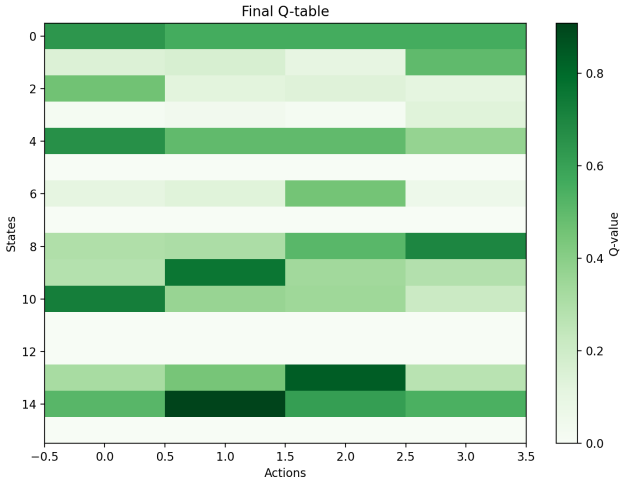
During training, the Q-values get more accurate, reflecting the agent's improved estimates of state–action quality. The progression from early, nearly uniform values to a clear separation of optimal actions illustrates the agent's learning over the episodes.



(a) Q-table after 500 episodes



(b) Q-table after 2000 episodes



(c) Final Q-table after 5000 episodes

Figure 4: Q-table across training at 500, 2000, and 5000 episodes.

Section 2 - Deep Q-Learning

Question 1: Experience Replay - Why do we sample in random order?

The experience replay technique addresses two major challenges in reinforcement learning. First, it prevents the model from forgetting previously learned behaviors, which can occur if the agent continues training only on recent experiences while older, useful samples are no longer seen. Second, sampling experiences in random order breaks the strong correlations between consecutive samples that arise during sequential training. This randomness helps the network learn more independently from past transitions, leading to more stable convergence and better generalization.

Question 2: Use of an older set of weights to compute the targets. How does this improve the model?

Using an older set of network weights for computing target Q-values helps stabilize the training process. If the same network were used for both prediction and target computation, the Q-value estimates would shift simultaneously, creating a moving target and increasing training instability. By freezing the target network temporarily, the model avoids this issue, allowing the prediction network to converge more smoothly. After several training steps, the target network is updated to the latest weights, ensuring stable and consistent learning.

DQN Implementation

In this section we implemented a Deep Q-Network (DQN) agent for the `CartPole-v1` environment using PyTorch. The state consists of 4 continuous variables and the action space has 2 discrete actions. We tested two fully-connected architectures:

- **3-layer DQN:** input layer of size 4, three hidden layers of size 128, and an output layer of size 2.
- **5-layer DQN:** input layer of size 4, five hidden layers of size 128, and an output layer of size 2.

The hyperparameters sweep was carried out over the following:

$$\alpha \in \{10^{-4}, 10^{-5}\}, \qquad \gamma \in \{0.99, 0.999\},$$

$$\text{batch\_size} \in \{64, 128\}, \qquad \text{target\_update\_period} \in \{100, 200\}$$

while keeping the following parameters fixed:

$$\varepsilon_{\max} = 1.0, \qquad \varepsilon_{\min} = 0.01, \qquad \varepsilon_{\text{decay}} = 0.999$$

For each configuration and for both network architectures (3-layer and 5-layer), a full training run was performed, and the mean reward over the last 100 episodes was recorded. Figure 5 shows the mean reward evolution for all tested configurations, while Figure 6 highlights the top five performing hyper-parameter settings. From this sweep, we selected the following hyper-parameters for the final solution:

$$\alpha = 10^{-4}, \qquad \gamma = 0.99, \qquad \text{batch\_size} = 128, \qquad \text{target\_update\_period} = 100.$$

Among the tested parameters, the learning rate α and discount factor γ had the greatest influence on performance: smaller α or larger γ led to slower learning and higher variance, whereas the selected values produced stable and relatively fast convergence. The batch size mainly affected the smoothness of the learning curve, and the target update controlled the balance between training stability and responsiveness to new information. Using the selected hyper-parameters, the 3-layer network reached an average reward of approximately 475 over 100 consecutive episodes after about $N \approx 460$ episodes, while the deeper 5-layer network solved the task slightly earlier at around $N \approx 420$ episodes. The loss and reward curves for the final 3 and 5 layers are summarized in Figures 7–11.

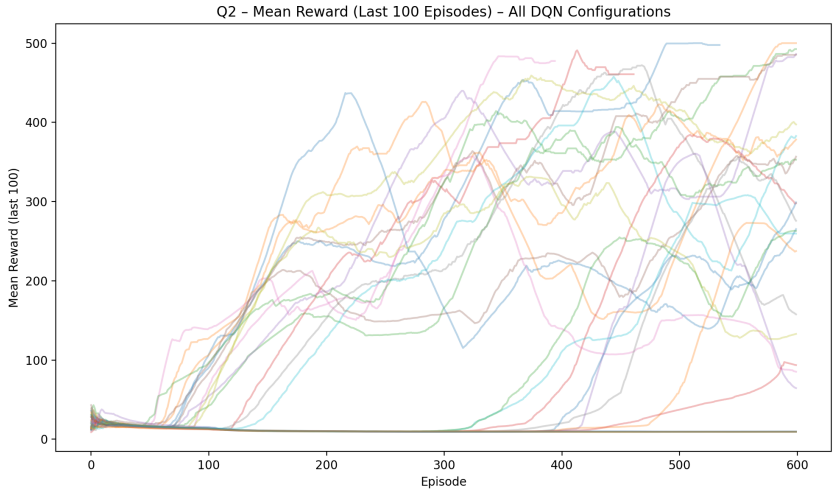


Figure 5: Mean reward (last 100 episodes) for all tested DQN hyper-parameter configurations.

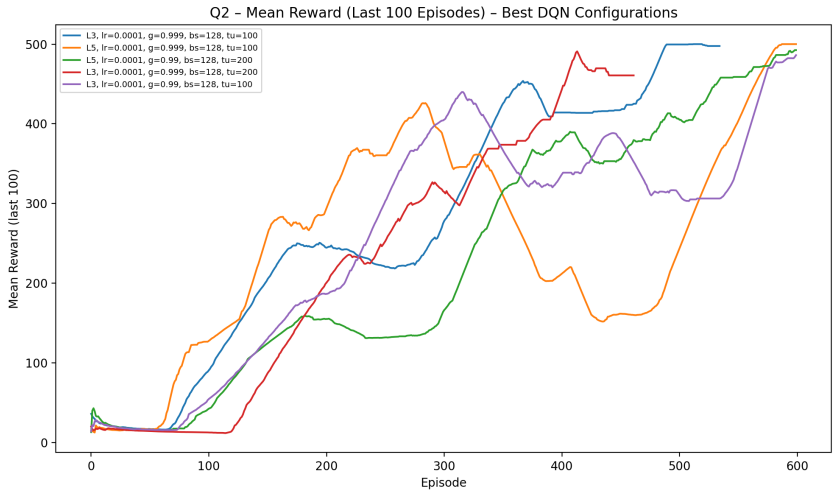


Figure 6: Mean reward (last 100 episodes) for the best five DQN configurations.

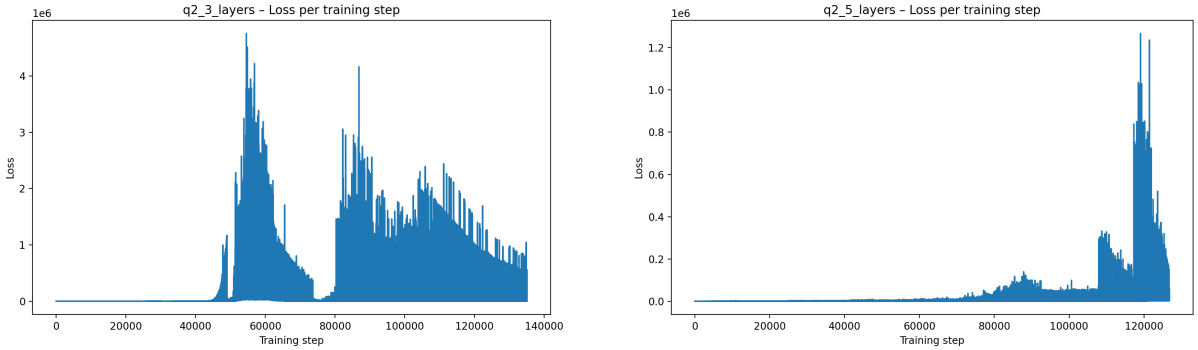


Figure 7: Loss per training step for the 3-layer (left) and 5-layer (right) DQN.

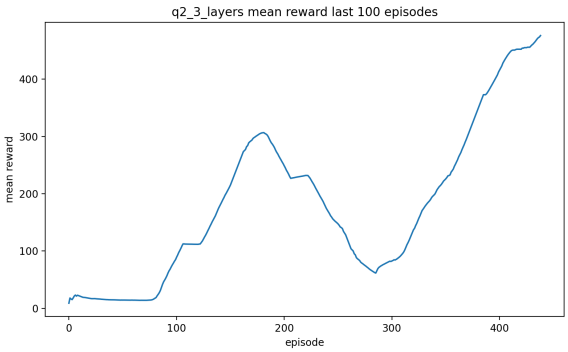


Figure 8: 3-layer DQN mean reward over the last 100 episodes.

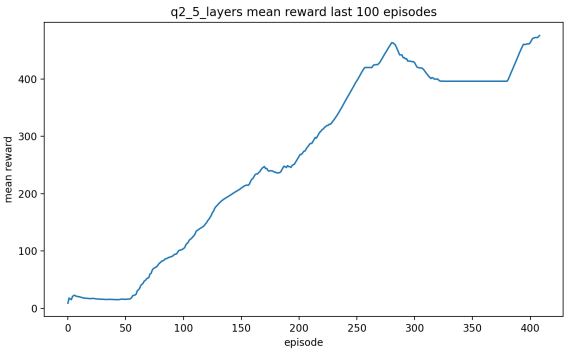


Figure 9: 5-layer DQN mean reward over the last 100 episodes.

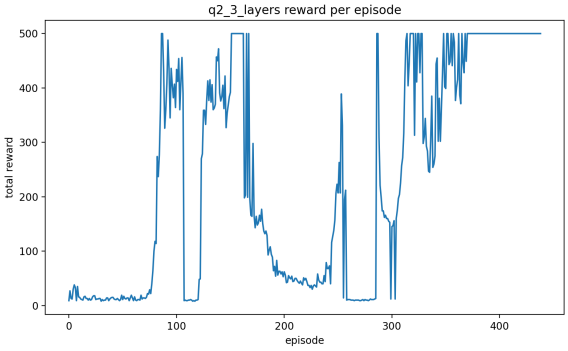


Figure 10: 3-layer DQN total reward per episode.

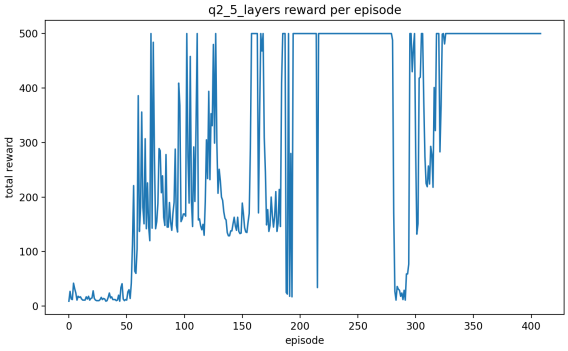


Figure 11: 5-layer DQN total reward per episode.

Section 3 – Improved DQN

For the improvement of the DQN we chose to combine two well-known algorithms, Double DQN (DDQN) and Dueling DQN. DDQN reduces overestimation bias by decoupling action selection and action evaluation, while the Dueling network structure separates state-value estimation from advantage estimation, improving learning efficiency in states where the choice of action matters less.

Double DQN Update

observe transition (s, a, r, s')
if UPDATE(A) **then**
    $a^* \leftarrow \arg\max_a Q^A(s', a)$
    $y \leftarrow r + \gamma Q^B(s', a^*)$
    $Q^A(s, a) \leftarrow Q^A(s, a) + \alpha(y - Q^A(s, a))$
else
    $b^* \leftarrow \arg\max_a Q^B(s', a)$
    $y \leftarrow r + \gamma Q^A(s', b^*)$
    $Q^B(s, a) \leftarrow Q^B(s, a) + \alpha(y - Q^B(s, a))$
end if

Instead of using the same network for both selection and evaluation, DDQN mitigates overestimation errors and yields more stable learning. The Dueling network decomposes the Q-value into a state-value term and an action-dependent advantage term:

$$Q(s, a) = V(s) + \Big(A(s, a) - \tfrac{1}{|A|} \sum_{a'} A(s, a')\Big).$$

This helps the agent learn meaningful value estimates even in states where all actions behave similarly, improving sample efficiency.

Python implementation

The Dueling DDQN agent is implemented on top of the DQN framework from Section 2, using the same replay buffer and overall training loop. Training was performed on `CartPole-v1` for up to 600 episodes. The only modifications are the DDQN target update shown above, and replacing the plain network with a Dueling network. We kept the best hyper-parameters found in Section 2:

$$\alpha = 10^{-4}, \qquad \gamma = 0.99, \qquad \text{batch\_size} = 128, \qquad \text{target\_update\_period} = 100,$$

$$\varepsilon_{\max} = 1.0, \qquad \varepsilon_{\min} = 0.01, \qquad \varepsilon_{\text{decay}} = 0.999.$$

Figure 12 shows the loss evolution for the 3-layer Dueling DDQN network. The loss remains small for most of training, with short spikes when the policy adjusts. The episodic reward curve (Figure 12) rises steadily and becomes consistently high once the agent stabilizes. The running mean reward in Figure 13 highlights the improvement relative to the 3-layer DQN from Section 2. While the standard DQN first surpassed the 475-reward threshold only around **episode 460**, the Dueling DDQN agent reaches this level much earlier, at approximately **episode 320**. Beyond converging faster, the Dueling DDQN curve is noticeably smoother and exhibits fewer oscillations, indicating reduced overestimation bias and more stable value estimates throughout training.

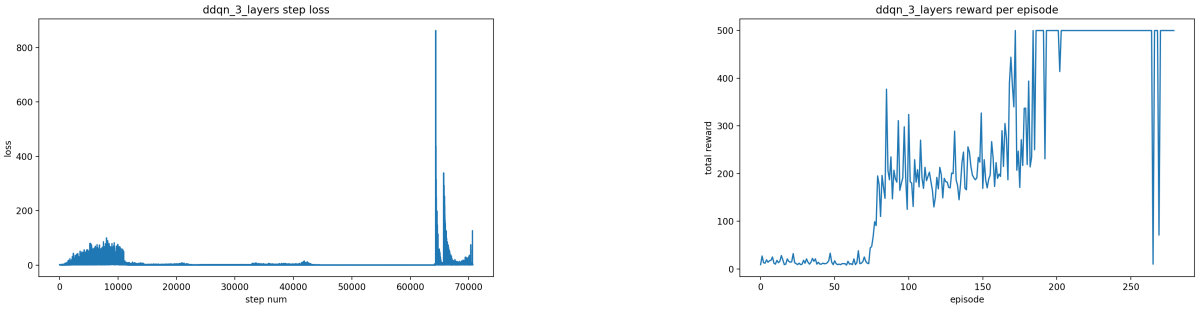


Figure 12: 3-layers Dueling DDQN Loss per training step (left) and total reward per episode (right).

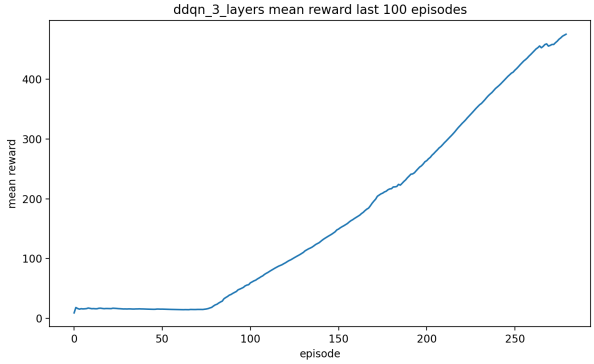


Figure 13: 3-layers Dueling DDQN mean reward over the last 100 episodes.

Appendix

Code Execution Explanation

To run the submitted scripts, a `conda` environment file (`drl_hw1_env.yaml`) is provided. Create and activate the environment by running the following commands from the same folder the `yaml` file is located:

```
conda env create -f drl_hw1_env.yaml
conda activate drl_hw1_env
```

After activating the environment, navigate to the folder containing the Python files and run the wanted script:

- `q1_tabular_q_learning.py` runs section 1.
- `q2_dqn_cartpole.py` runs section 2.
- `q3_ddqn_cartpole.py` runs section 3.